

Here, the addresses of the variables `x`, `y` and `z` are passed as argument and not variables. Then in the function definition those addresses are assigned to the integer pointers `m`, `n` and `o`, respectively. Here in function definition the variables are accessed with a `*` operator with the pointers mentioned above.

5.4. Return By Reference

Besides the call by reference method, a function can also return a reference. Consider the following example.

```
int & minimum ( int& x, int& y)
{
    if (x<y)
        return x;
    else
        return y;
}
```

Here a function `minimum` is declared which is of the return type `int&`. Due to this, a reference to the variables `x` and `y` is returned. Hence, if another function called `min` is declared, then it will refer to the variables declared inside the `minimum` function.

5.5 Inline Functions

With a concept known as 'inline function', the time required for calling small functions is drastically reduced. This reduces execution time. In general terms an inline function is a function that is expanded in line when it is invoked. It is expanded at compile time. The main syntax for declaring an inline function is as follows:

```
inline type name ( arguments ...) { instructions
... }
```